Documentación: MVC, MVP y MVVM

Este documento describe la arquitectura de los tres proyectos de ejemplo (`productos-mvc` , `productos-mvp` y `productos-mvvm`). Para cada uno se incluye:

- Un **diagrama de paquetes**, que muestra qué capa depende de cuál.
- Un **diagrama de clases**, que muestra las clases, sus miembros principales y sus relaciones.

Los diagramas están escritos en PlantUML. Para verlos puedes usar la extensión de PlantUML de VS Code o IntelliJ, o pegarlos en https://www.plantuml.com/plantuml.

Las clases del dominio, `Producto` y `ProductoRepositorio` , son iguales en las tres aplicaciones. Lo único que cambia es la organización de la capa de presentación.

---

1. MVC: Model-View-Controller

1.1 Diagrama de paquetes

```
@startuml mvc-paquetes
title MVC – Diagrama de paquetes
set separator none
skinparam packageStyle rectangle

package "pe.edu.ulima.is2.mvc" {
  package "modelo" as modelo {
  }
  package "vista" as vista {
  }
  package "controlador" as controlador {
  }
  class App
}

package "javax.swing" as swing <<JDK>> {
}

App ..> modelo
App ..> vista
App ..> controlador

vista ..> modelo : observa y lee
vista ..> controlador : reenvía eventos
vista ..> swing
controlador ..> modelo : modifica

note bottom of modelo
  El modelo no depende de
  ningún otro paquete.
end note
@enduml
```

1.2 Diagrama de clases

```
@startuml mvc-clases
title MVC – Diagrama de clases
skinparam classAttributeIconSize 0
hide empty members

package modelo {
  class Producto <<record>> {
    + id : int
    + nombre : String
    + categoria : String
    + precio : double
    + stock : int
  }

  class ProductoRepositorio {
    - productos : List<Producto>
    + listarTodos() : List<Producto>
    + buscar(texto : String) : List<Producto>
  }

  class ProductoModelo {
    - repositorio : ProductoRepositorio
    - observadores : List<Observador>
    - filtro : String
    - productos : List<Producto>
    + agregarObservador(o : Observador) : void
    + filtrar(texto : String) : void
    + getFiltro() : String
    + getProductos() : List<Producto>
    - notificar() : void
  }

  interface "ProductoModelo.Observador" as Observador {
    + modeloActualizado() : void
  }
}

package controlador {
  class ProductoControlador {
    - modelo : ProductoModelo
```

```
    + buscar(texto : String) : void
    + limpiar() : void
  }
}

package vista {
  class ProductoVista {
    - modelo : ProductoModelo
    - txtBusqueda : JTextField
    - btnBuscar : JButton
    - btnLimpiar : JButton
    - lblEstado : JLabel
    - modeloTabla : DefaultTableModel
    + setControlador(c : ProductoControlador) : void
    + modeloActualizado() : void
  }
}

class App {
  + {static} main(args : String[]) : void
}

class JFrame <<javax.swing>>

ProductoRepositorio "1" o-- "*" Producto
ProductoModelo --> "1" ProductoRepositorio
ProductoModelo o-- "*" Observador : notifica >
ProductoModelo ..> Producto

ProductoVista --|> JFrame
ProductoVista ..|> Observador
ProductoVista --> ProductoModelo : lee estado >
ProductoVista ..> ProductoControlador : reenvía eventos >

ProductoControlador --> ProductoModelo : modifica >

App ..> ProductoModelo : <<create>>
App ..> ProductoVista : <<create>>
App ..> ProductoControlador : <<create>>
App ..> ProductoRepositorio : <<create>>
@enduml
```

Puntos clave

- `ProductoVista` implementa `Observador`. Cuando el modelo cambia, la vista **lee los datos directamente** de `ProductoModelo` y da formato a los precios.
- `ProductoControlador` solo modifica el modelo. **No tiene ninguna referencia a la vista.**
- `ProductoModelo` conoce a sus observadores solo a través de la interfaz `Observador`, así que no depende del paquete `vista`.

---

2. MVP: Model-View-Presenter

2.1 Diagrama de paquetes

```
@startuml mvp-paquetes
title MVP – Diagrama de paquetes
set separator none
skinparam packageStyle rectangle

package "pe.edu.ulima.is2.mvp" {
  package "modelo" as modelo {
  }
  package "vista" as vista {
  }
  package "presentador" as presentador {
  }
  class App
}

package "javax.swing" as swing <<JDK>> {
}

App ..> modelo
App ..> vista
App ..> presentador

presentador ..> modelo : consulta
presentador ..> vista : usa la interfaz ProductoVista
vista ..> presentador : reenvía eventos
vista ..> swing

note "La vista NO depende del modelo" as N1
N1 .. vista
@enduml
```

2.2 Diagrama de clases

```
@startuml mvp-clases
title MVP – Diagrama de clases
skinparam classAttributeIconSize 0
hide empty members

package modelo {
  class Producto <<record>> {
    + id : int
    + nombre : String
    + categoria : String
    + precio : double
    + stock : int
  }

  class ProductoRepositorio {
    - productos : List<Producto>
    + listarTodos() : List<Producto>
    + buscar(texto : String) : List<Producto>
  }
}

package presentador {
  class ProductoPresentador {
    - vista : ProductoVista
    - repositorio : ProductoRepositorio
    + iniciar() : void
    + alBuscar() : void
    + alLimpiar() : void
    - mostrar(productos : List<Producto>) : void
  }
}

package vista {
  interface ProductoVista {
    + getTextoBusqueda() : String
    + setTextoBusqueda(texto : String) : void
    + mostrarProductos(filas : List<String[]>) : void
    + mostrarEstado(mensaje : String) : void
  }
```

```
  class ProductoVentana {
    - txtBusqueda : JTextField
    - btnBuscar : JButton
    - btnLimpiar : JButton
    - lblEstado : JLabel
    - modeloTabla : DefaultTableModel
    + setPresentador(p : ProductoPresentador) : void
    + getTextoBusqueda() : String
    + setTextoBusqueda(texto : String) : void
    + mostrarProductos(filas : List<String[]>) : void
    + mostrarEstado(mensaje : String) : void
  }
}

class App {
  + {static} main(args : String[]) : void
}

class JFrame <<javax.swing>>

ProductoRepositorio "1" o-- "*" Producto

ProductoPresentador --> "1" ProductoVista : actualiza >
ProductoPresentador --> "1" ProductoRepositorio : consulta >
ProductoPresentador ..> Producto : formatea

ProductoVentana ..|> ProductoVista
ProductoVentana --|> JFrame
ProductoVentana ..> ProductoPresentador : reenvía eventos >

App ..> ProductoVentana : <<create>>
App ..> ProductoPresentador : <<create>>
App ..> ProductoRepositorio : <<create>>
@enduml
```

Puntos clave

- `ProductoPresentador` depende de la **interfaz** `ProductoVista`, no de Swing. Por eso se puede probar con una vista falsa o cambiar la interfaz gráfica sin tocarlo.
- `ProductoVentana` es una vista pasiva: no importa nada del paquete `modelo` y recibe las filas ya formateadas como `List<String[]>`.

- El presentador **le indica a la vista** qué mostrar, llamando a `mostrarProductos` y `mostrarEstado`.

---

3. MVVM: Model-View-ViewModel

3.1 Diagrama de paquetes

```
@startuml mvvm-paquetes
title MVVM – Diagrama de paquetes
set separator none
skinparam packageStyle rectangle

package "pe.edu.ulima.is2.mvvm" {
  package "modelo" as modelo {
  }
  package "viewmodel" as viewmodel {
  }
  package "vista" as vista {
  }
  package "binding" as binding {
  }
  class App
}

package "javax.swing" as swing <<JDK>> {
}

App ..> modelo
App ..> viewmodel
App ..> vista

vista ..> viewmodel : se enlaza a
vista ..> binding : usa Enlazador
vista ..> swing
viewmodel ..> modelo : consulta
viewmodel ..> binding : expone Propiedad y Comando
binding ..> swing : Enlazador

note "El viewmodel NO depende de la vista ni de Swing" as N1
N1 .. viewmodel
@enduml
```

3.2 Diagrama de clases

```
@startuml mvvm-clases
title MVVM – Diagrama de clases
skinparam classAttributeIconSize 0
hide empty members

package modelo {
  class Producto <<record>> {
    + id : int
    + nombre : String
    + categoria : String
    + precio : double
    + stock : int
  }

  class ProductoRepositorio {
    - productos : List<Producto>
    + listarTodos() : List<Producto>
    + buscar(texto : String) : List<Producto>
  }
}

package binding {
  class "Propiedad<T>" as Propiedad {
    - valor : T
    - observadores : List<Consumer<T>>
    + get() : T
    + set(nuevoValor : T) : void
    + observar(o : Consumer<T>) : void
  }

  interface Comando <<functional>> {
    + ejecutar() : void
  }

  class Enlazador <<utility>> {
    + {static} texto(campo : JTextField, p : Propiedad<String>) : void
    + {static} etiqueta(etiqueta : JLabel, p : Propiedad<String>) : void
    + {static} comando(boton : AbstractButton, c : Comando) : void
    + {static} comando(campo : JTextField, c : Comando) : void
    + {static} tabla(modelo : DefaultTableModel, p : Propiedad<List<T>>, aFila
```

```
: Function<T, Object[]>) : void
  }
}

package viewmodel {
  class ProductoItem <<record>> {
    + id : String
    + nombre : String
    + categoria : String
    + precio : String
    + stock : String
  }

  class ProductoViewModel {
    - repositorio : ProductoRepositorio
    - textoBusqueda : Propiedad<String>
    - productos : Propiedad<List<ProductoItem>>
    - estado : Propiedad<String>
    - buscar : Comando
    - limpiar : Comando
    + textoBusqueda() : Propiedad<String>
    + productos() : Propiedad<List<ProductoItem>>
    + estado() : Propiedad<String>
    + comandoBuscar() : Comando
    + comandoLimpiar() : Comando
    - buscar() : void
    - limpiar() : void
    - cargar(lista : List<Producto>) : void
  }
}

package vista {
  class ProductoVista {
    - txtBusqueda : JTextField
    - btnBuscar : JButton
    - btnLimpiar : JButton
    - lblEstado : JLabel
    - modeloTabla : DefaultTableModel
    - enlazar(vm : ProductoViewModel) : void
    - construirInterfaz() : void
  }
}
```

```
class App {
  + {static} main(args : String[]) : void
}

class JFrame <<javax.swing>>

ProductoRepositorio "1" o-- "*" Producto

ProductoViewModel --> "1" ProductoRepositorio : consulta >
ProductoViewModel *-- "3" Propiedad
ProductoViewModel *-- "2" Comando
ProductoViewModel ..> ProductoItem : <<create>>
ProductoViewModel ..> Producto : transforma

ProductoVista --|> JFrame
ProductoVista ..> ProductoViewModel : data binding >
ProductoVista ..> Enlazador : usa >
Enlazador ..> Propiedad : observa / actualiza
Enlazador ..> Comando : ejecuta

App ..> ProductoViewModel : <<create>>
App ..> ProductoVista : <<create>>
App ..> ProductoRepositorio : <<create>>
@enduml
```

Puntos clave

- `ProductoViewModel` expone su **estado** como `Propiedad<T>` y sus **acciones** como `Comando`. No tiene ninguna referencia a la vista y no importa Swing.
- `ProductoVista` no tiene métodos del tipo `mostrarXxx()`. Solo conecta sus componentes al ViewModel en `enlazar()`, usando `Enlazador`.
- La flecha va **solo de la vista hacia el ViewModel**. En MVP, en cambio, el presentador también apunta a la vista a través de su interfaz.
- El paquete `binding` reemplaza lo que en JavaFX, WPF o Android ofrece la propia plataforma.

---

4. Comparación de dependencias

```
@startuml comparacion
title Dirección de las dependencias en cada patrón
left to right direction
skinparam packageStyle rectangle

package "MVC" {
  rectangle "Vista" as V1
  rectangle "Controlador" as C1
  rectangle "Modelo" as M1
  V1 --> C1 : eventos
  C1 --> M1 : modifica
  V1 --> M1 : observa y lee
}

package "MVP" {
  rectangle "Vista" as V2
  rectangle "Presentador" as P2
  rectangle "Modelo" as M2
  V2 --> P2 : eventos
  P2 --> V2 : ordena qué mostrar\n(vía interfaz)
  P2 --> M2 : consulta
}

package "MVVM" {
  rectangle "Vista" as V3
  rectangle "ViewModel" as VM3
  rectangle "Modelo" as M3
  V3 --> VM3 : data binding
  VM3 --> M3 : consulta
}
@enduml
```

Pregunta	MVC	MVP	MVVM
¿La vista conoce al modelo?	Sí	No	No
¿El intermediario conoce a la vista?	No	Sí, vía interfaz	No
¿Quién da formato a los datos?	Vista	Presentador	ViewModel
¿Cómo se actualiza la vista?	Observer sobre el modelo	El presentador llama a sus métodos	Data binding